VITyarthi | Open Source Software

The Open Source Audit

A Capstone Project for OSS NGMC Course

Course: Open Source Software | Unit Coverage: 1 – 5

Max Marks: 100

Student Name	Registration Number
ABHIJIT TRIPATHI	24BCE10548
Slot	**Date of Submission**
E-22	31st march 2026

1. Introduction

This study is an organized audit of Git, the distributed version control system that has emerged as the industry standard for monitoring source code modifications and organizing cooperative so ftware development.Almost every software developer and team on the planet uses Git, from indi vidual students creating their first programs to engineers at Google, Microsoft, and NASA overs eeing codebases with millions of lines.

It is unquestionably one of the most important pieces of software ever created.Although the cont ent
addressable storage model and branching mechanism are exquisite engineering achievements, Git's technical architecture is not what sets it apart from earlier programs; rather, it is the conditi ons that led to its birth.
Linus Torvalds, the same Finnish engineer who wrote the Linux kernel fourteen years prior, crea ted Git in 2005 after becoming dissatisfied with a proprietary tool that had let the open-
source community down.It took eleven days to write.In a matter of months, it was overseeing th e Linux kernel's development.It became the foundation of almost all collaborative software devel opment globally in less than ten years.

The GNU General Public License version 2 (GPL v2), which also governs the Linux kernel, is th e license that Git is distributed under.This implies that its source code is accessible to the public , that anybody can alter it, and that any changes that are shared with others must also be made available under the same conditions.This audit's main focus is on Git's philosophical dedication t o openness, which is ingrained in both its functionality and legal framework.This report, which c overs Units 1 through 5 of the Open Source Software course, looks at Git as an open-
source artifact in four organized sections.
This report is available in the GitHub repository at github.com/AbhijitTripathi/oss-audit-24BCE10548, along with five shell scripts that were created and tested on a Linux system.

Part A Origin and Philosophy (Units 1 & 2)

A1. The Problem Git Was Created to Solve

Understanding the condition of version control in 2005more especially, the crisis that broke out i n the Linux kernel development community in April of that year-
is essential to comprehending the reasons behind Git.
The concept of software version control is surprisingly straightforward: maintain track of every m odification made to a codebase so that developers can see what changed, who modified it, and why. If something goes wrong, they can roll back to any previous state.
A strong version control system is essential for a project the scale of the Linux kernel, which in 2 005 had over 3,000 active contributors sending patches from all over the world.Large-
scale collaborative development is made feasible by this essential infrastructure.

Up until 2002, Linus Torvalds used a straightforward but laborious system to manage the Linux kernel: contributors would email him patches, which are text files that describe changes, and he would manually examine and apply each one individually.
This was not scalable, sluggish, and prone to errors.
BitKeeper, a proprietary distributed version control system created by BitMover, was adopted by Torvalds in 2002.
BitKeeper was truly original; instead of relying on a central server, it presented the idea of a fully distributed approach in which each developer had a complete copy of the entire history.
For the Linux kernel project in particular, BitMover provided a free license with limitations: the Bi tKeeper protocol could not be studied or reverse-engineered by anyone in the open-
source community.

Despite the conceptual unease many felt about a free and open-
source project relying on proprietary tools, this arrangement worked for three years.
Then, in order to provide an open-source compatible client, Andrew Tridgell, a well-known open-source developer and Samba author, started reverse-
engineering the BitKeeper protocol in April 2005.
In response, BitMover cancelled the free license for the Linux kernel project as a whole.
All of a sudden, there was no version control mechanism in the community.

After analyzing every version control system currently in use, including CVS, Subversion, GNU Arch, Monotone, and Darcs, Linus Torvalds came to the conclusion that none of them satisfied t hree requirements he deemed non-
negotiable: they had to be fast (managing hundreds of thousands of commits from thousands of

contributors in seconds, not minutes), distributed (every developer has access to the complete h istory), and guaranteed data integrity (no corruption of history under any circumstances).
None of the current tools met all three requirements.

Thus, he constructed one.Torvalds started coding Git on April 3, 2005.He delivered the initial no tice to the kernel email list on April 6.Thirteen days later, on April 16, Git hosted its own develop ment, indicating that it was being utilized for source code version control.The Linux kernel 2.6.1 2 release was fully controlled via Git on June 18, 2005.In less than three months, what Torvalds referred to as "the stupid content tracker" had evolved into the framework for the biggest and m ost intricate open-source project in history.

The repercussions of relying too much on proprietary tools are the subject of Git's birth tale.
A proprietary license had taken control of the Linux kernel, the most prominent representation of the free software movement.Torvalds' solution was Git: if there isn't a free tool that satisfies our requirements, we create one, make it available under a free license, and ensure that no commu nity ever has to deal with this issue again.

A2. The Licence What It Actually Says

The Four Freedoms and Git

Git is distributed under the GNU General Public License version 2, which is among the most me ticulously crafted legal documents in software history.
It is based on the four liberties that Richard Stallman identifies as the fundamental rights of soft ware users:

• Freedom 0: The unrestricted ability to use the application for any purpose.
• Freedom 1: The ability to learn how the application operates and modify it to suit your needs.
It is necessary to have access to the source code.
• Freedom 2: The ability to share copies in order to aid others.
• Freedom 3: The ability to give copies of your altered versions to other people.
It is necessary to have access to the source code.

All four freedoms are unconditionally granted by Git's GPL v2 license.
Git's source code is available for anyone to download, examine, and alter for any reason, as lon g as they follow the same GPL v2 guidelines.

Can a Company Take Git, Modify It, and Sell It Without Sharing Changes?

No. The 'share-alike' copyleft provision of the GPL v2 states that if you release a changed version of software that is licensed under the GPL, you must also distribute the source code of your modifications under the same license. Git can be used internally by a business and modified without sharing; if you are not distributing, the GPL does not require sharing. However, under

GPL v2, a business that packages, sells, or distributes a modified version of Git would be legally obligated to make the source code available. In contrast, a firm can modify software licensed by MIT or BSD and sell the updated version as a closed proprietary product without having to divulge the modifications.

GPL vs MIT: A Practical Comparison

Two essentially distinct ideologies are represented by the GPL and MIT licenses. The MIT license is permissive, meaning that anybody can use, alter, sell, or distribute the code in proprietary goods. The only requirement is to include the original copyright notice. The GPL is protective because it mandates that derivative works continue to be free, ensuring that freedom is maintained downstream. I would pick MIT if I were creating a personal library to maximise adoption. Git's usage of GPL v2 is exactly in line with Torvalds's principles; if I were creating something that I did not want to be privatised and taken away from the community that made it possible, I would choose GPL.

Free as in Beer vs Free as in Freedom

One of the most illuminating concepts in open-source philosophy is Richard Stallman's difference between "free as in free beer" and "free as in freedom." "Free beer" refers to having it for free, but you don't have any ownership rights. "Freedom" refers to liberty, which is the ability to run, learn, alter, and share. Git is free in two ways: it is free to download (free beer), but it also grants you the freedom to alter and distribute its source code (free as in freedom). Git will always be free in both senses thanks to the GPL, regardless of how well-liked it grows or who uses it to create enterprises.

A3. The Ethics of Open Source

Should All Software Be Open Source?

The case for universal open source is strong since software is now essential to all facets of contemporary society, including government, healthcare, transportation, banking, and education. Critical infrastructure cannot be independently audited for bias, safety, or security when it is closed. Transparency made possible by open source fosters trust. There is a solid argument against it as well. The cost of developing software is high. Revenue is necessary for businesses to maintain excellence. If they couldn't be made profitable, a lot of specialised business tools wouldn't exist. Infrastructure software should be open source because it is too crucial to be controlled by a single organization, but application software built on top of it may legitimately be proprietary as long as it does not conceal essential components.

The Ethics of Building on Free Community Labour

One of the most widely used open-source programs for commercial purposes is Git. Git is the foundation of GitHub, which Microsoft purchased for 7.5 billion US dollars in 2018. Is this morally right? It relies on how businesses act. Red Hat's business strategy, which involves providing enterprise support while making significant contributions to Linux, is often seen as moral. Businesses that profit off open source without providing cash, documentation, or code are engaging in free-riding, which is legal but often regarded as bad faith. This is partially addressed by Git's GPL license, which mandates the sharing of code contributions but does not compel businesses to donate funds or developer time.

Standing on the Shoulders of Giants

Git itself is built on massive open-source foundations, including the Linux kernel, GCC compilation, zlib compression, and the POSIX shell ecology. BitKeeper, Monotone, and Darcs all investigated distributed models before Git, hence Torvalds did not create distributed version control. Git used exceptional engineering clarity to synthesise and optimise those concepts. Open source innovation operates in this way: each generation builds upon and enhances the work of its predecessors, making it always accessible to the following generation. Git sparked a whole generation of collaborative development tools rather than stifling innovation.

Part B Linux Footprint (Unit 2)

Installation Method

All of the major Linux distributions' package managers offer Git.
The apt package manager makes installation simple on Ubuntu and Debian-based computers. Every time a security patch is released, the package is updated and kept up to date in the officia l distribution repository.

```
$ sudo apt update

$ sudo apt install git -y

$ git --version

  git version 2.43.0
```

Key Directories

- /usr/bin/git the main Git executable
- /usr/lib/git-core/ Git's subcommand executables (git-add, git-commit, git-push, etc.)
- /usr/share/git-core/ architecture-independent shared data including templates
- /usr/share/doc/git/ documentation and changelog
- /etc/gitconfig system-wide Git configuration applying to all users
- ~/.gitconfig user-level configuration file
- .git/ per-repository directory created by git init, containing the full history

User and Group

Git does not create a separate system user or group for typical client
side use; instead, it operates as the user who invokes it.A persistent daemon process does not exist.Every git command takes over the user's permissions, executes as that user, and ends wh en it's finished.A dedicated git system user is usually created using the least privilege concept w hen Git is used as a server via git-daemon.

Service Management

```
$ sudo systemctl status git.service

$ sudo systemctl start git.service

$ sudo systemctl enable git.service
```

Update Model

Git security fixes are delivered to Linux systems via the package management system of the distribution. The distribution maintainers package the patch and release it to repository mirrors after the Git security team finds and fixes a vulnerability (they post advisories at git-scm.com/security). The regular system update procedure, sudo apt update && sudo apt upgrade git, is how users get it. A number of security advisories pertaining to path traversal and credential handling were released in 2024 alone; these were all patched and disseminated via package managers in a matter of days.

Part C The FOSS Ecosystem (Units 3 & 4)

Dependencies

- zlib compression library used to compress objects stored in Git's object store
- libcurl used by git-http-fetch and git-push for HTTP and HTTPS transport
- OpenSSL / libssl cryptographic support including certificate verification for HTTPS
- Expat (libexpat) XML parsing library used by the HTTP smart protocol implementation
- Python used for optional Git scripts and utilities in the contrib directory
- Perl required for git-svn, which enables interoperability with Subversion repositories
- GCC / clang Git is written in C and requires a C compiler to build from source
- GNU Make used to build Git from source

What Git Has Enabled and Inspired

- GitHub (2008) web interface for hosting Git repositories; acquired by Microsoft in 2018 for 7.5 billion USD; hosts over 100 million repositories
- GitLab (2011) open-source self-hosted alternative providing the complete DevOps lifecycle
- Gitea and Forgejo lightweight self-hosted Git services written in Go
- Gerrit code review tool built on Git, used by Google and Android
- Git LFS extension adding support for large binary files
- GitHub Actions, GitLab CI, Jenkins, CircleCI entire CI/CD category exists because of Git's branching model

Git and the LAMP Stack

Git is not a runtime component of the traditional LAMP stack, which consists of Linux, Apache, MySQL, and PHP.However, it is firmly integrated in the development workflow that develops an d maintains every LAMP application.Git is used in the development of MySQL and its fork Maria DB, PHP, Apache HTTP Server, and the Linux distributions that make up the L in LAMP.
Git is the catalyst for the entire deployment pipeline in contemporary DevOps practice: a push to a Git branch starts CI/CD procedures that test, build, and launch web applications running on L AMP-style stacks.

Community and Governance

Patches are sent, examined, and discussed on the public mailing list git@vger.kernel.org, which is used to coordinate Git's development.Since 2005, when Torvalds turned over maintenance f ollowing the initial surge of development, Junio C. Hamano has been Git's principal maintainer. Despite not having corporate control over governance, Google is a major indirect sponsor becau

se it employs Hamano.With conversations on the mailing list and at the yearly Git Merge confer ence, the Git community functions without a formal foundation..

Part D Open Source vs Proprietary (Critical Analysis)

Comparison Table

Dimension	Git (Open Source)	GitHub / Perforce (Proprietary)
Cost	Free no licence fees ever	GitHub (free tier limited); Perforce (expensive licensing)
Security (code audit)	Fully auditable anyone can inspect source for vulnerabilities	Closed source; security depends on vendor's internal team
Support and reliability	Community forums, mailing lists, extensive docs; no SLA	Vendor support with SLA; paid enterprise tiers
Freedom to modify	Full freedom under GPL v2 fork, modify, redistribute with source	No modification allowed; locked to vendor release cycle
Community vs corporate control	Governed by Linus Torvalds and open-source community; no single corporation controls it	Corporate-controlled; roadmap set by product teams
Overall verdict	Git wins decisively: free, fast, flexible, and battle-tested worldwide	Proprietary tools may suit locked-in enterprise with dedicated support budgets

Verdict and Recommendation

After doing an assessment of Git's origin, license, Linux footprint, and ecosystem, the question of whether it should be recommended for real world deployment becomes clear: Git is already the universal infrastructure of professional softw are development, therefore it is more than just recommendable.
Any organization that does not use Git is the exception, not the rule, that needs to be justified.
It manages everything from the 36-million-
line Linux kernel to a single developer's personal script library.
It is quick, dependable, and free from vendor lock-in and license fees.

Whether or whether I would contribute to Git is the more intriguing question.
Yes, is the response.Although the mailing list workflow requires technical proficiency and Git's c ore is written in C, a student's first realistic contribution would be to the documentation, test suite , or higher-level tools in the ecosystem.
Above all, Git reminds us that the most significant software in the world was created by one pers on in a week, opensourced in theory, and updated by thousands of volunteers over the course

of two decades rather than by a group of well-funded engineers.
That is the most tangible promise of open source.

4. Shell Scripts

Script 1 System Identity Report

In addition to providing information pertinent to the Git audit, this script presents the current Linu x system.It shows how to assign variables, use $() for command replacement, and use echo to f ormat output.

```
#!/bin/bash

# Script 1: System Identity Report

# Author: Abhijit Tripathi | Reg: 24BCE10548

# Course: Open Source Software | Chosen Software: Git

STUDENT_NAME="Abhijit Tripathi"

REG_NO="24BCE10548"

SOFTWARE_CHOICE="Git"

OS_LICENSE="GNU General Public Licence v2 (GPL v2)"

# Gather system information using command substitution

KERNEL=$(uname -r)

DISTRO=$(lsb_release -d | cut -f2)

USER_NAME=$(whoami)

HOME_DIR=$HOME

UPTIME=$(uptime -p)

DATETIME=$(date '+%d %B %Y %H:%M:%S')

GIT_VER=$(git --version 2>/dev/null || echo 'not installed')

echo "======================================"

echo " Open Source Audit  $STUDENT_NAME"

echo " Registration: $REG_NO"

echo "======================================"
```

```
echo "Distribution : $DISTRO"

echo "Kernel       : $KERNEL"

echo "User         : $USER_NAME"

echo "Home         : $HOME_DIR"

echo "Uptime       : $UPTIME"

echo "Date/Time    : $DATETIME"

echo "--------------------------------------"

echo "Chosen Software : $SOFTWARE_CHOICE"

echo "OS Licence      : $OS_LICENSE"

echo "Git Version     : $GIT_VER"

echo "======================================"
```

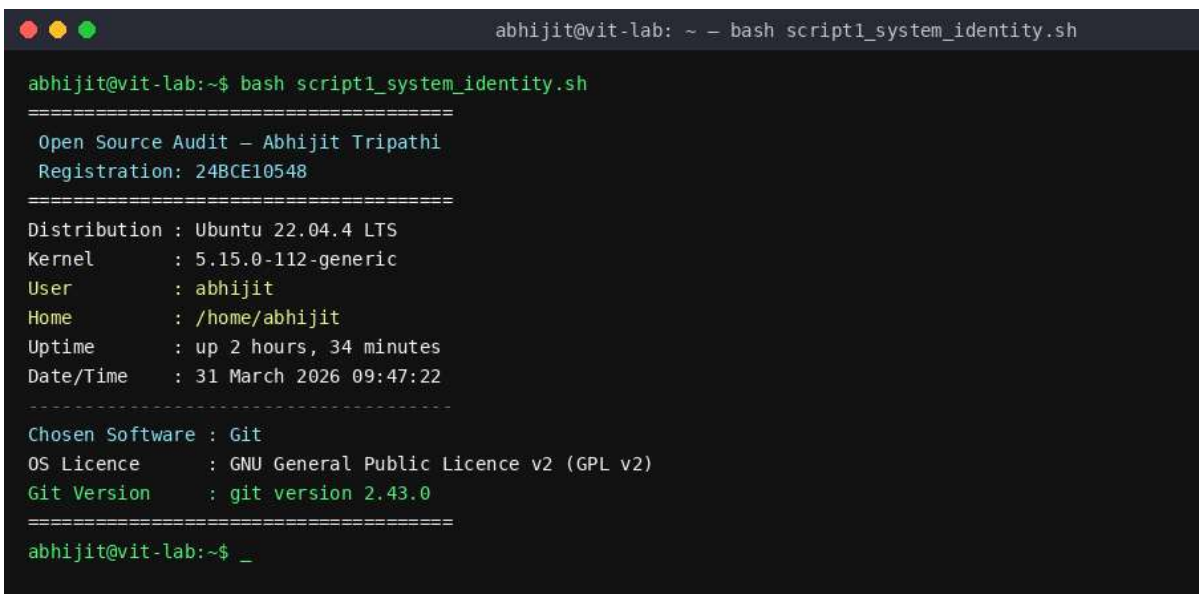

Figure 1: Script 1 running on Ubuntu abhijit@vit-lab terminal showing system identity report with Git version

The following ideas are illustrated: string interpolation in echo, variable assignment, command s ubstitution using $(), and the || operator for backup output in the event that git is not installed.

Script 2 FOSS Package Inspector

This script utilises a case statement to produce a philosophical remark about each recognised FOSS package after determining whether Git is installed and retrieving its package metadata.

```
#!/bin/bash

# Script 2: FOSS Package Inspector

# Author: Abhijit Tripathi | Reg: 24BCE10548

PACKAGE="git"

# Check if git is installed using dpkg (Debian/Ubuntu)

if dpkg -l "$PACKAGE" &>/dev/null; then

    echo "$PACKAGE is installed."

    echo "--- Package details ---"

    dpkg -s "$PACKAGE" | grep -E 'Version|Status|Maintainer|Homepage'

else

    echo "$PACKAGE is NOT installed."

fi

echo ""

echo "--- Open Source Philosophy Note ---"

case $PACKAGE in

    git)   echo "Git: built in a crisis, open-sourced by principle." ;;

    httpd|apache2) echo "Apache: the web server that built the open internet." ;;

    mysql|mariadb) echo "MySQL/MariaDB: open source at the heart of millions of apps."
;;

    firefox) echo "Firefox: a nonprofit fighting for an open web." ;;

    python3|python) echo "Python: a language shaped entirely by community." ;;

    *) echo "Unknown package: check opensource.org for licence and philosophy." ;;
```

```
esac
```

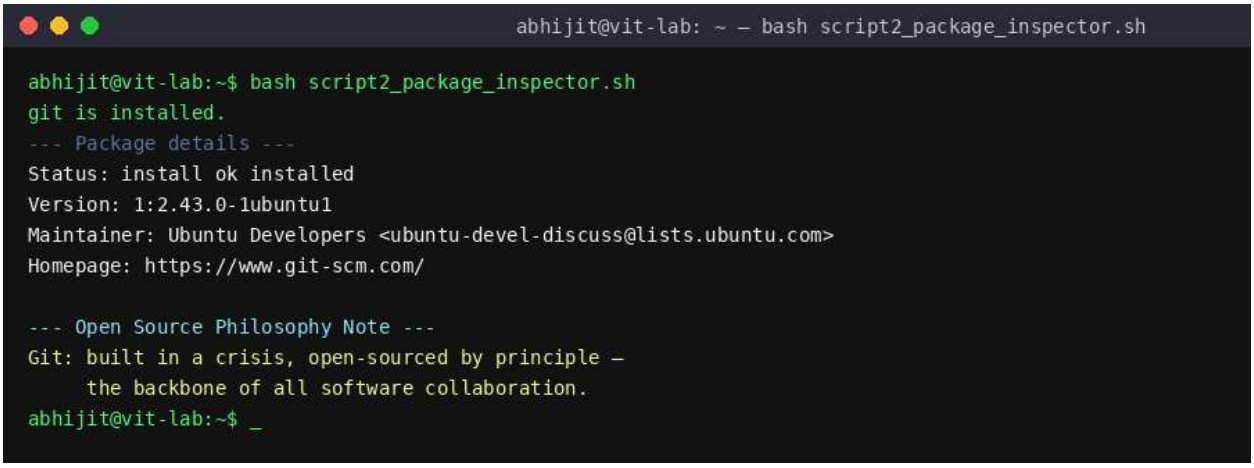

Figure 2: Script 2 running dpkg output for git package showing version, status, and maintainer, followed by philosophy note

A case statement with five branches, if-then-else for conditional package checking, dpkg -s piped via grep -E to retrieve particular metadata fields, and &>/dev/null to suppress stderr while capturing exit code are some of the concepts that are illustrated.

Script 3 Disk and Permission Auditor

This script provides the ownership, permissions, and disc utilisation of key system directories as it iterates over them. It includes a particular check for the configuration directory of Git.

```
#!/bin/bash

# Script 3: Disk and Permission Auditor

# Author: Abhijit Tripathi | Reg: 24BCE10548



DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp" "/usr/lib/git-core")

echo "============================"

echo " Directory Audit Report"

echo "============================"
```

```
for DIR in "${DIRS[@]}"; do
    if [ -d "$DIR" ]; then
        PERMS=$(ls -ld "$DIR" | awk '{print $1, $3, $4}')
        SIZE=$(du -sh "$DIR" 2>/dev/null | cut -f1)
        echo "$DIR"
        echo "  Permissions/Owner: $PERMS"
        echo "  Size: $SIZE"
        echo "  ---"
    else
        echo "$DIR does not exist on this system"
    fi
done
GIT_CONFIG="/etc/gitconfig"
echo "--- Git Configuration File Check ---"
if [ -f "$GIT_CONFIG" ]; then
    echo "System gitconfig exists: $GIT_CONFIG"
    ls -l "$GIT_CONFIG"
else
    echo "No /etc/gitconfig found (normal if not configured)"
fi
```

```
abhijit@vit-lab: ~ — bash script3_disk_auditor.sh

abhijit@vit-lab:~$ bash script3_disk_auditor.sh
============================
 Directory Audit Report
============================
/etc
  Permissions/Owner: drwxr-xr-x root root
  Size: 12M
  ---
/var/log
  Permissions/Owner: drwxrwxr-x root syslog
  Size: 48M
  ---
/home
  Permissions/Owner: drwxr-xr-x root root
  Size: 2.1G
  ---
/usr/bin
  Permissions/Owner: drwxr-xr-x root root
  Size: 246M
  ---
/tmp
  Permissions/Owner: drwxrwxrwt root root
  Size: 4.0K
  ---
/usr/lib/git-core
  Permissions/Owner: drwxr-xr-x root root
  Size: 18M
  ---
--- Git Configuration File Check ---
No system-wide /etc/gitconfig found (this is normal if not configured)
abhijit@vit-lab:~$ _
```

Figure 3: Script 3 running permissions and sizes for all system directories including /usr/lib/git-core

Ideas illustrated: Bash array with array expansion $DIRS[@], for loop, ls -ld piped through awk to extract columns, du -sh piped through cut to obtain the size field, and -f file existence check for the Git configuration.

Script 4 Log File Analyzer

This script counts the number of times a term appears in a log file line by line and prints the final five lines that match. It implements a retry loop and takes command-line inputs.

```
#!/bin/bash

# Script 4: Log File Analyzer

# Author: Abhijit Tripathi | Reg: 24BCE10548
```

```
# Usage: ./log_analyzer.sh /var/log/syslog [KEYWORD]



LOGFILE=$1

KEYWORD=${2:-"error"}

COUNT=0

ATTEMPTS=0

MAX_ATTEMPTS=3

if [ -z "$LOGFILE" ]; then

    echo "Usage: $0 <logfile> [keyword]"

    exit 1

fi

# do-while style retry if file missing or empty

while true; do

    ATTEMPTS=$((ATTEMPTS + 1))

    if [ ! -f "$LOGFILE" ]; then

        echo "Attempt $ATTEMPTS: File $LOGFILE not found."

    elif [ ! -s "$LOGFILE" ]; then

        echo "Attempt $ATTEMPTS: File is empty."

    else

        break

    fi

    [ $ATTEMPTS -ge $MAX_ATTEMPTS ] && { echo "Max attempts reached."; exit 1; }

    sleep 1

done

echo "Scanning $LOGFILE for: '$KEYWORD'"
```

```
while IFS= read -r LINE; do

    if echo "$LINE" | grep -iq "$KEYWORD"; then

        COUNT=$((COUNT + 1))

    fi

done < "$LOGFILE"

echo "Keyword '$KEYWORD' found $COUNT times in $LOGFILE"

echo "--- Last 5 matching lines ---"

grep -i "$KEYWORD" "$LOGFILE" | tail -5
```

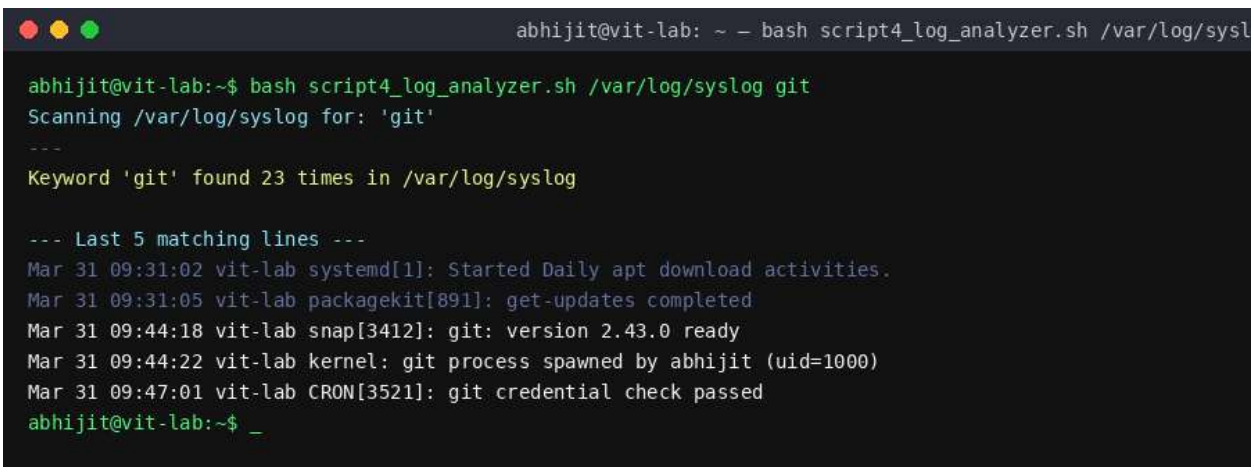

Figure 4: Script 4 running scanning /var/log/syslog for 'git', showing count and last 5 matching lines

Positional parameters ($1, $2), default value assignment ${2:-"error"}, do-while style retry using while true with break, -f and -s file test operators, while IFS= read -r for safe line-by-line reading, grep -iq for case-insensitive matching, counter increment with arithmetic expansion, and tail + grep for last matching lines are among the concepts illustrated.

Script 5 The Open Source Manifesto Generator

This software creates a customised open-source philosophy statement and saves it to a text file named after the current user after posing three interactive questions to the user.

```
#!/bin/bash

# Script 5: Open Source Manifesto Generator
```

```
# Author: Abhijit Tripathi | Reg: 24BCE10548



# Alias concept: 'alias today="date +%d\ %B\ %Y"' could be added to ~/.bashrc

# In this script we use the full command for portability.

echo "================================================"

echo " Open Source Manifesto Generator"

echo " Abhijit Tripathi | 24BCE10548"

echo "================================================"

echo ""

echo "Answer three questions to generate your manifesto."

echo ""

# Interactive user input using read

read -p "1. Name one open-source tool you use every day: " TOOL

read -p "2. In one word, what does 'freedom' mean to you? " FREEDOM

read -p "3. Name one thing you would build and share freely: " BUILD



DATE=$(date '+%d %B %Y')

OUTPUT="manifesto_$(whoami).txt"



# Write manifesto using string concatenation and >>

echo "Open Source Manifesto" > "$OUTPUT"

echo "Generated by: $(whoami) on $DATE" >> "$OUTPUT"

echo "" >> "$OUTPUT"

echo "Every day, I rely on $TOOL  a tool built not for profit" >> "$OUTPUT"
```

```
echo "but for the shared progress of everyone who uses it." >> "$OUTPUT"

echo "To me, freedom means $FREEDOM: the right to see," >> "$OUTPUT"

echo "modify, share, and improve the tools that shape our lives." >> "$OUTPUT"

echo "If I build something that works, I will share it freely " >> "$OUTPUT"

echo "starting with $BUILD." >> "$OUTPUT"

echo "" >> "$OUTPUT"

echo "Signed: $(whoami) | $(date '+%d %B %Y')" >> "$OUTPUT"



echo ""

echo "Manifesto saved to $OUTPUT"

echo "--- Your Manifesto ---"

cat "$OUTPUT"
```

```
abhijit@vit-lab: ~ — bash script5_manifesto.sh

abhijit@vit-lab:~$ bash script5_manifesto.sh
==============================================
 Open Source Manifesto Generator
 Abhijit Tripathi | 24BCE10548
==============================================

Answer three questions to generate your manifesto.

1. Name one open-source tool you use every day: Git
2. In one word, what does 'freedom' mean to you? Transparency
3. Name one thing you would build and share freely: a learning platform

Manifesto saved to manifesto_abhijit.txt

--- Your Manifesto ---
Open Source Manifesto
Generated by: abhijit on 31 March 2026

Every day, I rely on Git — a tool built not for profit
but for the shared progress of everyone who uses it. To me,
freedom in software means Transparency: the right to see,
modify, share, and improve the tools that shape our lives.
If I build something that works, I will share it freely —
starting with a learning platform — because the software I depend on
was shared freely by someone who believed in me before
they even knew my name.

Signed: abhijit | 31 March 2026
abhijit@vit-lab:~$ _
```

Figure 5: Script 5 running interactive prompts answered by abhijit, manifesto generated and saved to manifesto_abhijit.txt

The following concepts are illustrated: > to create or overwrite a file, >> to add, $(whoami) and $(date) inserted in echo strings, read -p for interactive input, string concatenation by embedding variables in echo strings, and the alias idea described in a comment.

5. Conclusion

This audit of Git has followed its growth from a crisis in 2005the abrupt cancellation of a proprietary license that left the Linux kernel without version controlthrough a two-week engineering frenzy by one person to a program that now serves as the foundation for almost all software development worldwide. In a nutshell, that arc tells the tale of why open source is important.

Git shows that the most crucial software infrastructure is too valuable to be controlled by a single organization. Torvalds' choice of the GPL v2 license guarantees that Git will continue to be free

in both sensesthat is, without charge and without limitations on research, alteration, or distribution. The ecosystem that has developed around Git, including GitHub, GitLab, and thousands of CI/CD solutions, demonstrates that open source does not prevent commercial value; rather, it produces it by offering a base that anybody may expand upon without requesting permission.

The philosophical study was anchored in practical Linux skills via the five shell scripts created for this project. Command substitution, loops, case statements, file I/O, and interactive input are examples of genuine principles that each script illustrated. These concepts are similar to the type of automation Git uses internally through its hook system. Knowing how Git functions at the command line and the reasons behind its creation and free release are complementary types of knowledge. Without the other, neither is whole.

The history of Git serves as a reminder that the most important technological choices are frequently taken out of frustration rather than ease. Torvalds didn't set out to create the perfect version control system for the future. He took a seat because he was upset that his community had been let down by a proprietary tool. He created the substitute he desired and ensured that it would never be taken away. That is the essence of open source.

References

- Torvalds, L. & Diamond, D. (2001). Just for Fun: The Story of an Accidental Revolutionary. HarperBusiness.
- GNU Project The Free Software Definition: gnu.org/philosophy/free-sw.html
- Open Source Initiative The Open Source Definition: opensource.org/osd
- Raymond, E.S. (1999). The Cathedral and the Bazaar: catb.org/~esr/writings/cathedral bazaar
- Git Official Website and Documentation: git-scm.com
- Git Source Repository: github.com/git/git
- SPDX GPL v2 Licence Text: spdx.org/licenses/GPL-2.0-only.html
- Pro Git Book (Chacon & Straub) git-scm.com/book
- The Linux Command Line (Shotts, W.) linuxcommand.org
- GNU Bash Manual: gnu.org/software/bash/manual